The router — how a route is found

`docs/web/route/router.js`, with `route.js` and `polar.js` under it. It reads the files described in `GRID.md` and `COURSE.md` and answers one question:

> With this boat, this wind and this stream, starting at this time, what is the earliest she could finish, and by what track?

It is not a tactician. It knows nothing of other boats, of sail changes, of a crew that has been up all night, or of the difference between a wind forecast and a wind. What it gives is a *bound* and a *shape*: no boat with that polar could have done better in that weather, and this is where the fastest water was.

Built from the bottom, in four layers

An isochrone router draws a plausible track from almost any arithmetic, which makes it one of the hardest kinds of program to tell is wrong. So each layer was finished and checked while it was still small enough to check by hand.

	layer	what it does	its check
1	`ROUTE.vec`, `advance`	the boat's vector plus the stream's; the position after it	`route-vector.js`
2	`POLAR.speed`, `sail`	the boat's speed at an angle and a wind speed	`polar.js`
3	`ROUTE.step`	one time step, from the grids and the polar	`route-step.js`
4	`ROUTER.run`	every heading from everywhere she could be, until one finishes	`router.js`

One step

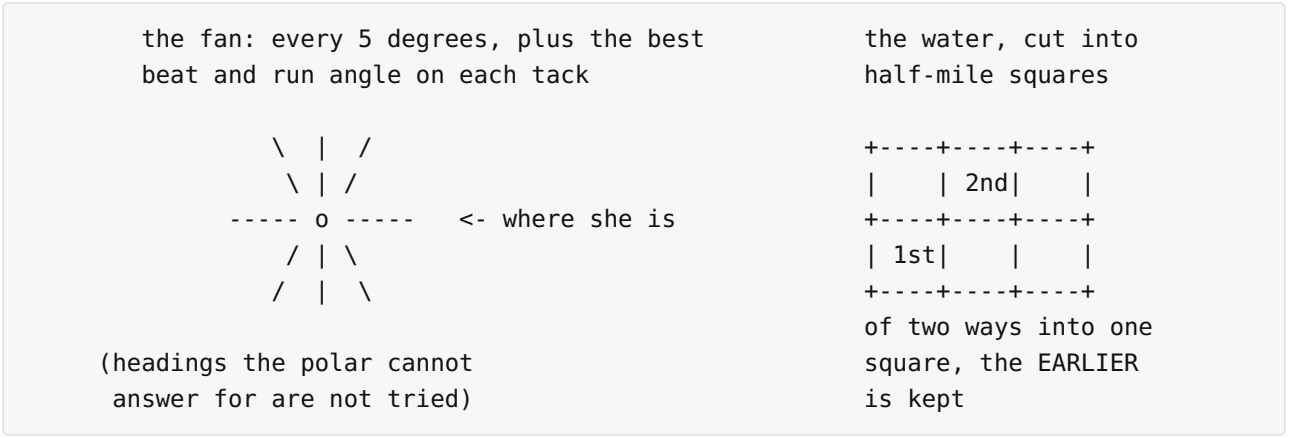
```
            wind over the ground, 12 kt from the west
      ------------------------------------------------>
      --------->  the stream, 2 kt, going the same way

      ----------------------------------------->
            wind over the WATER, 10 kt:  the boat's wind
```

The boat sails in the wind over the water. A polar is measured through the water, so it is entered with the ground wind less the stream. With the stream running *against* the wind the two add, which is why wind against tide is windier as well as rougher. Every row of a route carries both winds.

The polar then gives her speed through the water at her angle to that wind; the stream is added to it; and she is moved over the ground for the length of the step. The conditions are those at the start of the step, held for the whole of it.

Beating and running come from the polar file's own optimum rows. A table that starts at 52 degrees says nothing about 45, and a boat that could not sail closer than that could not beat. The file's `beat_angle` and `beat_vmg` rows say where the best upwind angle is and what it makes good, so `POLAR.sail` answers out to that angle and no further; the same deep downwind, where the boat gybes rather than runs square. Nothing is invented: every speed lies between two numbers that are in the file.

The search

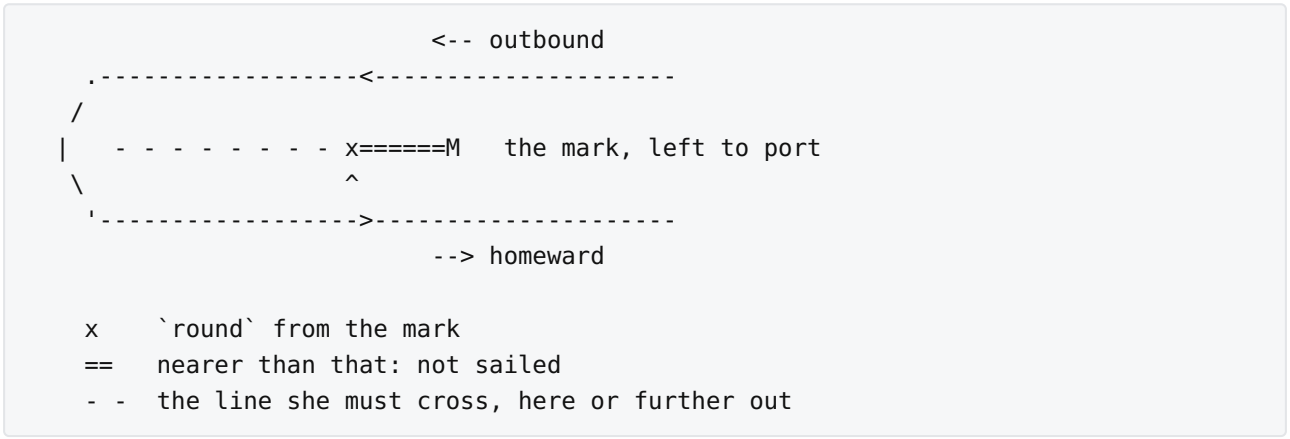


From every place she could be, every heading is tried for one time step. Of all the ways of reaching a half-mile square, only the earliest is kept; of two that arrive in the same step, the one nearer where she is bound. Then the same again from all the places kept, until a step crosses the finish line.

The one assumption in it: that being somewhere sooner is never worse than being there later. That is true of a boat that could always stop and wait, and near enough true of one that cannot. It is what makes the search finish, and it is the ordinary assumption of an isochrone router, stated here rather than left to be found.

When a mark or the finish is within a step, one more heading is tried: the one that lays it exactly. A finish line a tenth of a mile long would otherwise fall between two headings of the fan.

Rounding a mark



The racing rule is a string: drawn taut along the track, it must pass each mark on the required side. The router lays a line out from the mark through the middle of the turn, and she has rounded it when she crosses that line going the right way round — anticlockwise to leave it to port, clockwise for starboard. A track that cuts inside never crosses the line; one that passes on the wrong hand crosses it backwards and is refused.

`round` is a berth (Larry, 13 November 2026): she crosses that line at least `round` miles out and passes no nearer at any time. The fastest route passes exactly that far off. Boats in the real race do go inside what a prudent berth would be, and have for years; a run can be given a smaller number to see what that is worth, and nothing here is advice to do it.

A mark that is the end of the finish line is rounded by finishing, and its side is checked against the way the line is crossed.

The water she may not cross

in the files	what the router does
a `line` in a live layer	never crosses it, and stays the layer’s clearance off it
an `area` in a live layer	never enters it; its edges are lines
`avoid <name> ... off <nm>`	stays at least that far from the point
a layer the run does not name	is not in the water at all
the start line, on the first leg	is a wall behind her

A place on the start line that is already inside a clearance — one end of the 2025 line is a flagstaff ashore — is not a place a boat can start from, and is left out and counted.

What a route says about itself

Every row is one step: where and when it began, the heading and tack, both winds, the stream, the speed through the water and over the ground, and where it ended. The summary adds them up, and says three things a reader should know before trusting the rest:

`noStreamMinutes`	how long she sailed where the atlas has no data, through still water
`clampedMinutes`	how long the wind was above the strongest in the polar
`optimumMinutes`	how long she was on the polar's beat or run rows rather than its table

The first run: the 2025 Myth of Malham

Sailed on the course's own files — the GFS wind of 00Z on race day, the Shom stream atlas, the low-water coastline kept a tenth of a mile off, a generic 70 ft polar — in about three seconds.

Start, IRC One	10:00Z, 24 May, at the outer end of the line
The wind	248 at 16 knots at the start: **a dead beat to the Eddystone**
Eddystone, left to port	01:14Z, after 15 hours and 27 tacks, 0.13 miles off
Finish	09:42Z, 25 May: **23.7 hours**, 263 miles over the ground
Off Portland Bill	0.21 miles: inside the race, with the tide
Where the atlas is silent	the first 100 minutes, in the Solent

Checked afterwards by brute force, with none of the router's own bookkeeping: no step crosses the coastline, the nearest it comes is 0.146 miles, the traffic scheme is never approached, and the track swings 360 degrees anticlockwise round the lighthouse.

The real Clipper 68s took about 44 hours. The difference is the point, not a fault: this is the fastest a generic 70-footer could go, tacking for nothing, steered perfectly, in a forecast that came exactly true.

Staying a mile off the three headlands cost nothing the router can measure. The same race with `avoid` lines at St Alban's Head, Portland Bill and Start Point finishes within the same minute, a difference smaller than the search itself can resolve. On that day, in that wind, the inside passage at the Bill was not worth having.

What it does not do yet, and what it assumes

1. **A tack costs nothing.** She short-tacks the Solent every ten minutes because nothing says she should not.
2. **No gates.** `avoid` is sailed; `gate ... within` is refused with the reason rather than ignored.
3. **A mark that may be left on either hand** is refused likewise.
4. **The polar is the boat.** A generic polar gives a generic boat's passage.
5. **The forecast is taken as the truth,** and the stream atlas as the stream.
6. **It is not for navigation.** It is for understanding a race after it was sailed, and for asking what a different choice would have cost.

Running the checks

```
node docs/web/tests/route-vector.js     the vector step
node docs/web/tests/polar.js            the polar
node docs/web/tests/route-step.js       one step, against hand-worked answers
node docs/web/tests/router.js           whole routes small enough to sail in your
head
```

Each course in `router.js` is laid on the equator, in a wind and a stream that do not change, so that the fastest passage is one triangle and can be worked out: twenty miles on a beam reach at ten knots is two hours, and a dead beat is the distance over the best speed made good. Eight faults were put into the router one at a time — marks rounded the wrong way, no berth, clearances ignored, an area left open, the finish crossed either way, finishing without rounding, `avoid` dropped, unnamed layers left live — and each is caught.